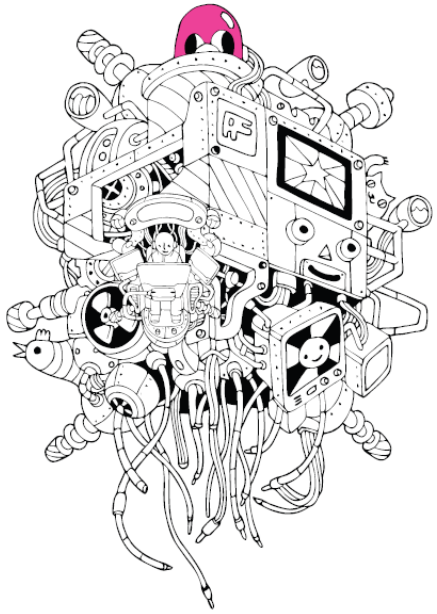
C programming



C Programming:

Chapter 05. Constants and Basic Data Types

C programming

Chapter 05-1. What the C language provides Understanding basic data types

C Programming:

A data type is a way to represent data.

Should I store real numbers or integers ?

· The way the value is stored differs depending on whether it is a real number or an integer, so the purpose must be determined .

How big a number will it store !

· A large number of bytes are required to express a large number .

In addition to the name, two pieces of information are needed to allocate me mory space:

Example of a request

" Ah ! I'm going to store an integer . I'm going to make it 4 bytes in size. That should be enough . And I'll name the variable num ."

A proper request

Example in

```
int num;
```

Same request

The number of data types refers to the number of ways to represent data. If the number of basic data types provided by the C language is 10, it means that the number of basic data representation methods provided by the C language is 10.

Types of basic data types and range of data expression

Data type		Data size	Data range
Integer type	char	1 byte	−128 ~ +127
	short	2 byte	−32,768 ~ +32,767
	int	4 byte	−2,147,483,648 ~ +2,147,483,647
	long	4 byte	−2,147,483,648 ~ +2,147,483,647
	long long	8 byte	−9,223,372,036,854,775,808 or more
			+9,223,372,036,854,775,807 or less
Real number type	float	4 byte	$\pm 3.4 \times 10^{-37}$ ~ $\pm 3.4 \times 10^{+38}$
	double	8 byte	$\pm 1.7 \times 10^{-307}$ ~ $\pm 1.7 \times 10^{+308}$
	long double	8 Byte or more	Greater than double

· **There may be slight differences depending on the compiler.**

The C standard only standardizes the relative sizes of data types, but does not mention specific sizes.

· **It is broadly divided into integer and real number types.**

Because the way data is represented is divided into two types: integer and real number !

· **There are two or more basic data types, both integer and real number .**

Multiple data types are provided so that you can choose the appropriate one depending on the size of the value you want to express !

Checking byte size using “sizeof” operator

```
int main(void)
{
    int num = 10;
    int sz1 = sizeof(num);
    int sz2 = sizeof(int);
    . . . .
}
```

By the size of the variables “num” and size of integer,

sz1 and sz2 are initialized

The operands of the sizeof operator can be variables , constants , and data type names.

Parentheses are required only for data type names like int!

However, it is common to surround all operands with parentheses !

```
int main(void)
{
    char ch=9;
    int inum=1052;
    double dnum=3.1415;
    printf("Size of variable ch: %d \n", sizeof(ch));
    printf("Size of variable inum: %d \n", sizeof(inum));
    printf("Size of variable dnum: %d \n", sizeof(dnum));

    printf("Size of char: %d \n", sizeof(char));
    printf("Size of int: %d \n", sizeof(int));
    printf("Size of long: %d \n", sizeof(long));
    printf("long long의 크기: %d \n", sizeof(long long));
    printf("Size of float: %d \n", sizeof(float));
    printf("Size of double: %d \n", sizeof(double));
    return 0;
}
```

Execution results

```
Size of variable ch: 1
Size of variable inum: 4
Size of variable dnum: 8
Size of char: 1
Size of int: 4
Size of long: 4
Size of long long: 8
Size of float: 4
Size of double: 8
```

General data type for representing and processing integers

· **A common choice is int.**

The CPU to calculate is determined by the size of the int type.

When an operation is performed, the type is converted to int and the operation is performed.

Therefore, it is appropriate to declare a variable that involves operations as int.

· **Are char type and short type variables unnecessary?**

If you need to store a large amount of data without involving any computation (or minimal computation),

And if the size of the data can be sufficiently expressed as char or short,

It is appropriate to represent and store data as char or short.

```
int main(void)
{
    char num1=1, num2=2, result1=0;
    short num3=300, num4=400, result2=0;

    printf("size of num1 & num2: %d, %d \n", sizeof(num1), sizeof(num2));
    printf("size of num3 & num4: %d, %d \n", sizeof(num3), sizeof(num4));
    printf("size of char add: %d \n", sizeof(num1+num2));
    printf("size of short add: %d \n", sizeof(num3+num4));

    result1=num1+num2;
    result2=num3+num4;
    printf("size of result1 & result2: %d, %d \n", sizeof(result1), sizeof(result2));
    return 0;
}
```

+ The reason of the size of the operation result is 4 bytes is because the operand was converted to 4 byte data.

Execution results

```
size of num1 & num2: 1, 1
size of num3 & num4: 2, 2
size of char add: 4
size of short add: 4
size of result1 & result2: 1, 2
```

General data type for representing and handling real numbers

· **The selection criteria for real number data types is precision.**

The range of real numbers that can be expressed is wide enough for both float and double.

However, double which is 8 bytes in size, represents real numbers with more precision than float.

· **The common choice is double.**

With the advancement of computing environments, the representation and operation of double type data has become less burdensome. float type data is often insufficient.

Real data type	Decimal precision	Number of bytes
float	6 digits	4
double	15 digits	8
long double	18 digits	12

```
int main(void)
{
    double rad;
    double area;
    printf("Enter the radius of the circle: ");
    scanf("%lf", &rad);
    area = rad*rad*3.1415;
    printf("Area of circle: %f \n", area);
    return 0;
}
```

Output format character for double type variable %f - printf

Input format character %lf for double type variables - scanf

Execution results

```
Enter circle radius: 2.4
Area of the circle: 18.095040
```

Unsigned

Integer Type	Size (bytes)	Range
char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295
long long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long	8 bytes	0 to 18,446,744,073,709,551,615

- **unsigned declaration can be placed in front of the name of an integer data type.**
- **When unsigned is attached , the MSB is also used to express the size of the data.**
- **Therefore, the range of values expressed is limited to positive integers and doubles as a positive integer.**

C programming

Chapter 05-2. Expression method of characters and data types for characters

C Programming:

A convention for representing characters! ASCII code

ascii code	ASCII code value
A	65
B	66
C	67
`	96
~	126

American National Standards Institute (ANSI) enacted by
'ASCII (American Standard Code for Information Interchange) code'

Computers cannot represent and store characters. Therefore, to represent characters, they assign a unique number to each character.
The characters entered by humans are converted into numbers and stored and recognized by the computer.
Numbers stored in a computer are converted into characters and displayed to the human eye.

```
int main(void)
{
    char ch1 = 'A';
    char ch2 = 'C';
    . . . .
}
```

```
int main(void)
{
    char ch1 = 65;
    char ch2 = 67;
    . . . .
}
```

At compile time, each character is converted to the corresponding ASCII code value.

So the data that is actually transmitted to the computer is numbers, not characters.

C programs, characters are expressed by enclosing them in single quotes. ' '

How characters are expressed!

```c
int main(void)
{
    char ch1='A', ch2=65;
    int ch3='Z', ch4=90;

    printf("%c %d \n", ch1, ch1);
    printf("%c %d \n", ch2, ch2);
    printf("%c %d \n", ch3, ch3);
    printf("%c %d \n", ch4, ch4);
    return 0;
}
```

Format character %c :

Print the ASCII code character of the corresponding number !

```
A 65
A 65
Z 90
Z 90
```

Execution results

The reason for storing characters in a char type variable

All ASCII code characters can be sufficiently expressed with 1 byte.

Characters do not involve operations such as addition or subtraction. They are only used for expression.

Therefore, a char type variable with a size of 1 byte is the optimal place to store characters.

Characters can also be stored in int type variables.

C programming

Chapter 05-3. Understanding Constants

C Programming:

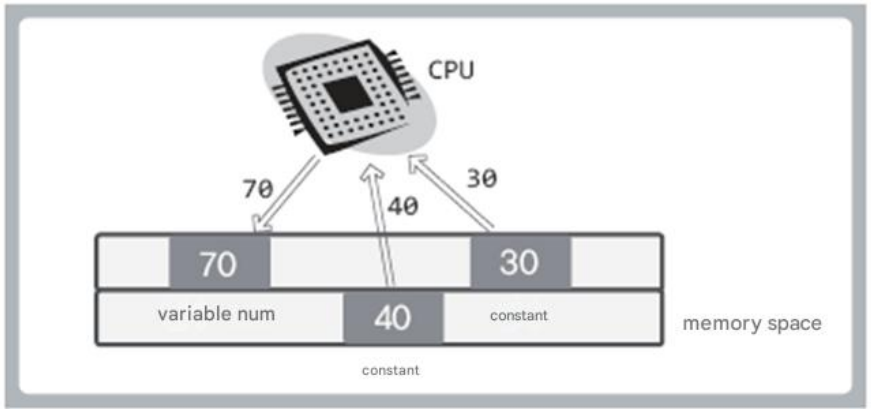
Literal constants without names!

```
int main(void)
{
    int num = 30 + 40;
    . . . .
}
```

For calculations, numbers expressed in the program, such as 30, 40 must be stored in memory space.

The value saved in this way is a constant that cannot be changed because it has no name.

Therefore, it is called a literal constant.



Step 1. Integers 30 and 40 are stored in memory space as constants.

Step 2. Addition is performed based on two constants.

Step 3. The integer 70 obtained as the result of the addition is stored in the variable num.

It must be stored in memory space to become a target of CPU calculation.

Data type of literal constant

```
int main(void)
{
    printf("literal int size: %d \n", sizeof(7));
    printf("literal double size: %d \n", sizeof(7.14));
    printf("literal char size: %d \n", sizeof('A'));
    return 0;
}
```

Execution results

```
literal int size: 4
literal double size: 8
literal char size: 4
```

Literal constants must have a determined data type before they can be stored in memory space.

The execution result of the above example implies the following facts :

- **Integers are basically represented as int type.**
- **Real numbers are basically expressed as double type.**
- **Constant characters are basically represented as int type.**

Character constants are evaluated as type int, and the actual storage size is determined by the variable type.

Expression of various constants using suffixes

```
int main(void)
{
    float num1 = 5.789;     // Warning
    float num2 = 3.24 + 5.12;     // Warning
    return 0;
}
```

Real numbers are recognized as double data type

A warning message about data loss appears.

```
float num1 = 5.789f;        // No warning
float num2 = 3.24F + 5.12F;     // Can use F or f
```

The data type of a constant can be changed through a suffix.

Integer

suffix	data type	example of use
U	unsigned int	unsigned int n = 1025U
L	long	long n = 2467L
UL	unsigned long	unsigned long n = 3456UL
LL	long long	long long n = 5768LL
ULL	unsigned long long	unsigned long long n = 8979ULL

Real number

suffix	data type	example of use
F	float	float f = 3.15F
L	long double	long double f = 5.789L

Floating-point constants are double by default, so assigning them to a float may cause a precision loss warning.

Symbolic constants with names : const constants

```
int main(void)
{
    const int MAX=100;          // MAX is a constant! Therefore, the value cannot be changed!
    const double PI=3.1415;     // PI is a constant! Therefore, the value cannot be changed!
    . . . .
}
```

```
int main(void)
{
    const int MAX;      // Initialized with garbage values
    MAX=100;            // Value cannot be changed! Therefore, a compilation error occurs!
    . . . .
}
```

Usually, constant names are all capitalized and,

When connecting two or more words, it is customary to separate the two words using an underscore, such as MY_AGE !

C programming

Chapter 05-4. Data Type Conversion

C Programming:

Automatic type conversion that occurs during assignment operations

Type conversion occurs based on the left side of the assignment operator.

```
double num1=245; // Automatic type conversion of int integer 245 to double
int num2=3.1415; // Automatic type conversion of double floating-point number 3.1415 to int
```

The integer 245 is reconstructed into a bit string of 245.0 and stored in the variable num1.

The real number 3.1415 is reconstructed as int type data 3 and stored in the variable num2.

```
int num3=129;
char ch=num3;   // The value stored in the int variable num3 is automatically type-converted to char
```

3 bytes of the 4- byte data stored in the 4- byte variable num3 are lost and stored in the variable ch .

00000000 00000000 00000000 10000001 ➡ 10000001

Summary of automatic type conversion methods

Type-by-type summary of type conversion methods

- **Convert integer to real number** — **3 is 3.0 and 5 is 5.0**
- **Convert real numbers to integers** — **The values after the decimal point are lost.**
- **Converting a large integer to a small integer** — **The upper bytes are discarded to fit the size of the small integer .**

```
int main(void)
{
    double num1=245;
    int num2=3.1415;
    int num3=129;
    char ch=num3;

    printf("Integer 245 as a floating-point number: %f \n", num1);
    printf("Real number 3.1415 to integer: %d \n", num2);
    printf("Larger integer 129 to smaller integer: %d \n", ch);
    return 0;
}
```

int 129
0000 0000 0000 0000
0000 0000 1000 0001 (binary)

char 1000 0001 == -127

Execution results

```
Integer 245 as a floating-point number: 245.000000
Real number 3.1415 to an integer: 3
Large integer 129 to the small integer: -127
```

Automatic type conversion by integer promotion

In general, the integer data type that is most suitable for the CPU to process is defined as int.

Therefore, the speed of int type operations is the same or faster than the speed of operations of other data types .

So, in the following way

Type conversion occurs.

```
int main(void)
{
    short num1=15, num2=25;
    short num3=num1+num2;      // num1 and num2 are type-casted to int
    . . . .
}
```

Calculated with integer, stored with short

This is called ' integral promotion ' .

Automatic type conversion caused by data type mismatch of operands

```
double num1 = 5.15 + 19;
```

The data types of the two operands must match.

If they do not match, automatic type conversion occurs to make them match .

Based on the automatic type conversion rules below, the int type data 19 is converted to the double type data 19.0.

Addition is performed after type conversion .

In arithmetic operations

Automatic type conversion rules

Data types with larger byte sizes are given priority.

Real numbers are given priority over integers.

This is a standard to minimize data loss.

Explicit type conversion : forced type conversion

```
int main(void)
{
    int num1=3, num2=4;
    double divResult;
    divResult = num1 / num2;
    printf("Division result: %f \n", divResult);
    return 0;
}
```

num1 and num2 are integers, only the quotient is returned.

Execution results

```
Division result: 0.000000
```

```
divResult = (double)num1 / num2;
```

(double) means type conversion to double.

Explicit type conversion of num1 to double and automatic arithmetic type conversion during the / operation between num1 and num2 ! As a result, real number division is performed, and 0.75 is stored in divResult .

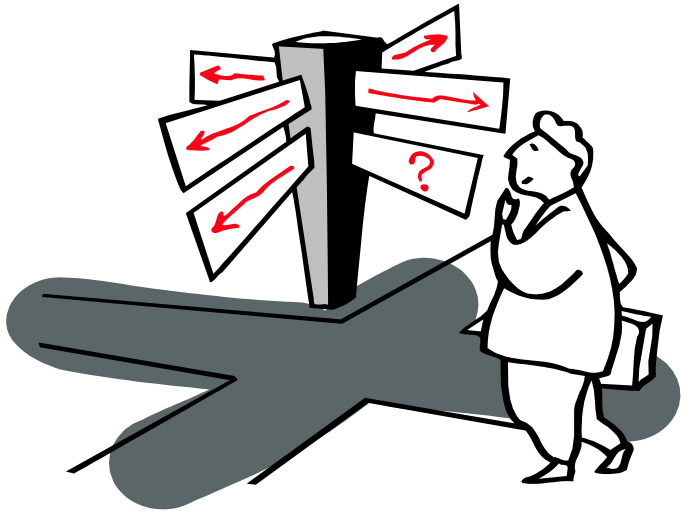
```
int main(void)
{
    int num1 = 3;
    double num2 = 2.5 * num1;
    . . . .
}
```

⟺

```
int main(void)
{
    int num1 = 3;
    double num2 = 2.5 * (double)num1;
    . . . .
}
```

It is a good idea to indicate that a type conversion is occurring by explicitly indicating the type conversion at the location where automatic type conversion occurs !

Recommended code writing style



Chapter 05 That's it . Do you have any questions ?